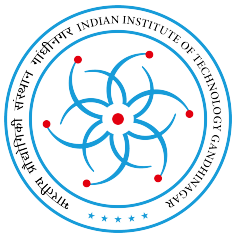

Indian Institute of Technology

Department of Computer Science

CS202: Software Tools and Technologies
Assessment 2

Jaskirat Singh Maskeen and Aarsh Wankar

(23110146, 23110003)

Instructor: Prof. Shouvick Mondal

October 18, 2025

Contents

Lab 6

Evaluation of Vulnerability Analysis Tools using CWE-based Comparison

Github Repository for Lab 6

6.1 Introduction

In this lab we evaluate and compare different static application security testing (SAST) tools based on their ability to detect software weaknesses in real-world, large-scale open-source projects. This comparison is based on the Common Weakness Enumeration (CWE), a community-developed list of the Top 25 most dangerous software and hardware weaknesses. By analyzing the CWEs reported by each tool, we can see their respective strengths and weaknesses, as well as their overall coverage of the top 25 CWEs. Furthermore, we will use metrics such as Intersection over Union (IoU) to quantify the pairwise agreement and disagreement between the tools, helping us understand their similarity and diversity in vulnerability detection.

6.2 Setup and Tools

We have Git, Python, and VSCode installed. `git --version` gives `git version 2.45.2.windows.1` as the output. The operating system on which all commands and analysis are performed is Windows 11. SEART Search Engine `https://seart-ghs.si.usi.ch/` is used to search and filter the repository based on our criteria. The working directory is named `STT_Lab_6`.

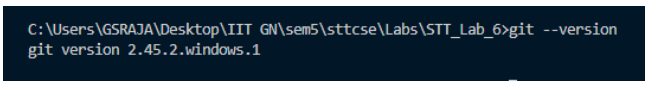

Figure 6.1: Git Version

6.3 Methodology and Execution

We used the tool SEART Search Engine to search for repositories. The selection criteria for these repositories is as follows Fig. 6.2:

- Language should be Java.
- There should be a minimum of 2 branches and 10 releases. (So that we can see the evolution of the code).
- The repository should have at least 10000 stars, indicating its popularity and activeness, and atmost 30000 stars, to avoid extremely large repositories.

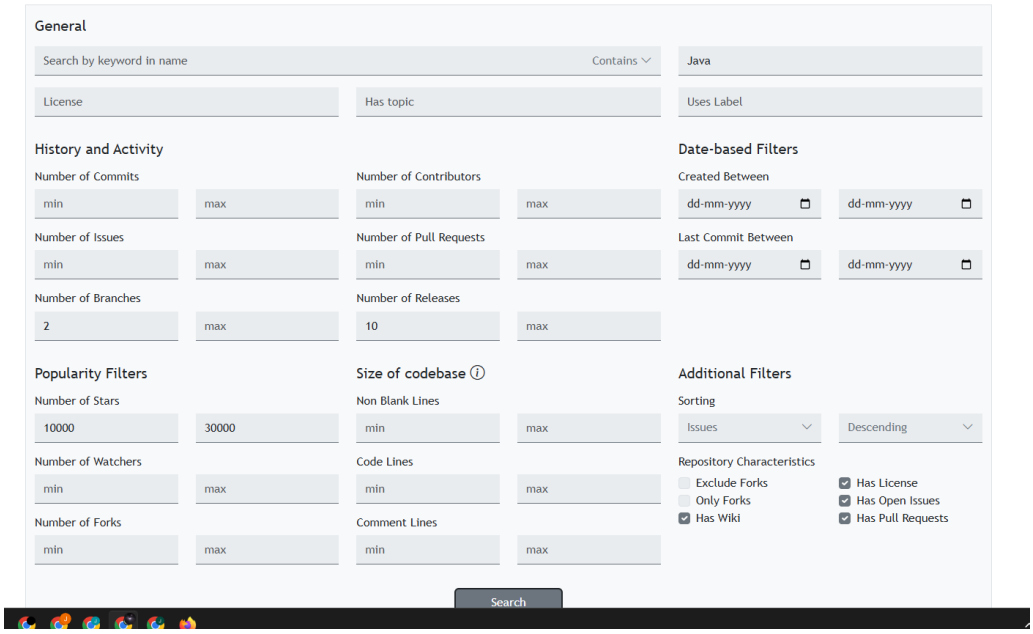

Figure 6.2: Search Criteria

- The repositories should have a Wiki, License, Open Issues, and Pull Requests.

Based on the results, we selected three large-scale, popular open-source repositories: **dagger**, **guice**, and **sentinel**.

The following three static analysis tools, which explicitly support CWE-based vulnerability reporting, were chosen for the evaluation:

- CodeQL: A powerful semantic code analysis engine.
- Semgrep: A fast, open-source static analysis tool for finding bugs and enforcing code standards.
- Snyk: A developer security platform that helps to find and fix vulnerabilities in code, open source dependencies, containers, and infrastructure as code.

Each of the three selected tools were run on all three chosen projects. The results of each scan were exported in the SARIF (Static Analysis Results Interchange Format), which is a structured JSON format. These SARIF files are available in the GitHub repository for this lab.

```
C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/alibaba/sentinel
Cloning into 'sentinel'...
remote: Enumerating objects: 27926, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 27926 (delta 23), reused 7 (delta 7), pack-reused 27894 (from 3)
Receiving objects: 100% (27926/27926), 8.18 MiB | 10.89 MiB/s, done.
Resolving deltas: 100% (10877/10877), done.
Updating files: 100% (1605/1605), done.

C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/google/guice
Cloning into 'guice'...
remote: Enumerating objects: 236640, done.
remote: Total 236640 (delta 0), reused 0 (delta 0), pack-reused 236640 (from 1)
Receiving objects: 100% (236640/236640), 122.37 MiB | 16.88 MiB/s, done.
Resolving deltas: 100% (199199/199199), done.

C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/google/dagger
Cloning into 'dagger'...
remote: Enumerating objects: 108403, done.
remote: Counting objects: 100% (145/145), done.
remote: Compressing objects: 100% (77/77), done.
remote: Total 108403 (delta 79), reused 75 (delta 62), pack-reused 108258 (from 4)
Receiving objects: 100% (108403/108403), 168.72 MiB | 13.83 MiB/s, done.
Resolving deltas: 100% (74025/74025), done.
Updating files: 100% (3419/3419), done.
```

Figure 6.3: Cloning the chosen repositories.

The setup for semgrep involved installing It via pip and setting an API token (from the website) as an environment variable followed by logging in via the command line (Fig. 6.4). For snyk, we installed it

via npm, and then set up authentication using CLI (Fig. 6.5). CodeQL was set up by downloading the CLI from the official website and configuring it in the PATH.

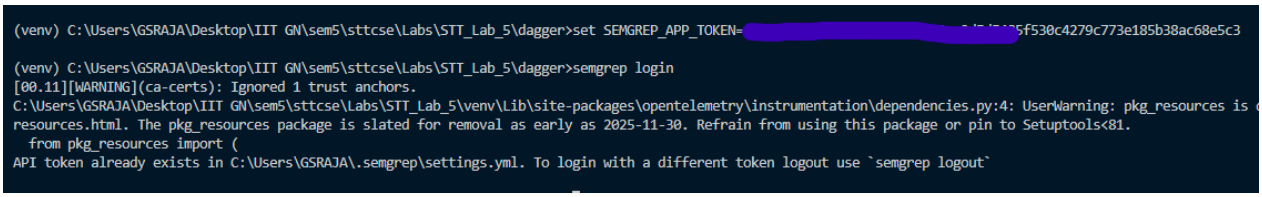

Figure 6.4: Semgrep Setup

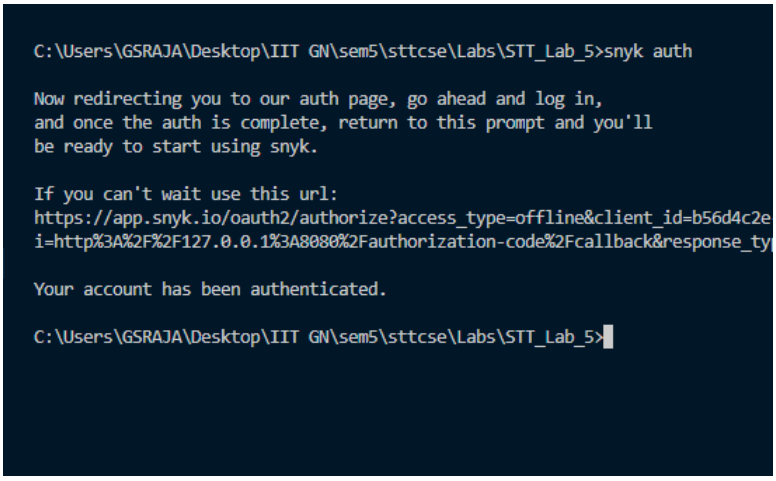

Figure 6.5: Snyk Setup

We then had to build some of the repositories, for example, dagger and guice, using `./gradlew build` command (Fig. 6.6).

Figure 6.6: Gardle Build for dagger

After building, we ran the tools on the repositories. For all three repositories (example here, dagger), the commands used were as follows:

- **CodeQL**: `codeql database create ../dagger-db -language=java-kotlin -build-mode=autobuild`
  followed by
  `codeql database analyze ../dagger-db codeql/java-queries:codeql-suites/java-security-and-quality.qls -format=sarif-latest -output=codeql_dagger-results.sarif`
- **Semgrep** (Fig. 6.7): `semgrep -config=auto ../dagger -json -output=semgrep_results_dagger.sarif`

- **Snyk** (Fig. 6.8): `snyk code test -sarif-file-output=../snyk_results_dagger.sarif`

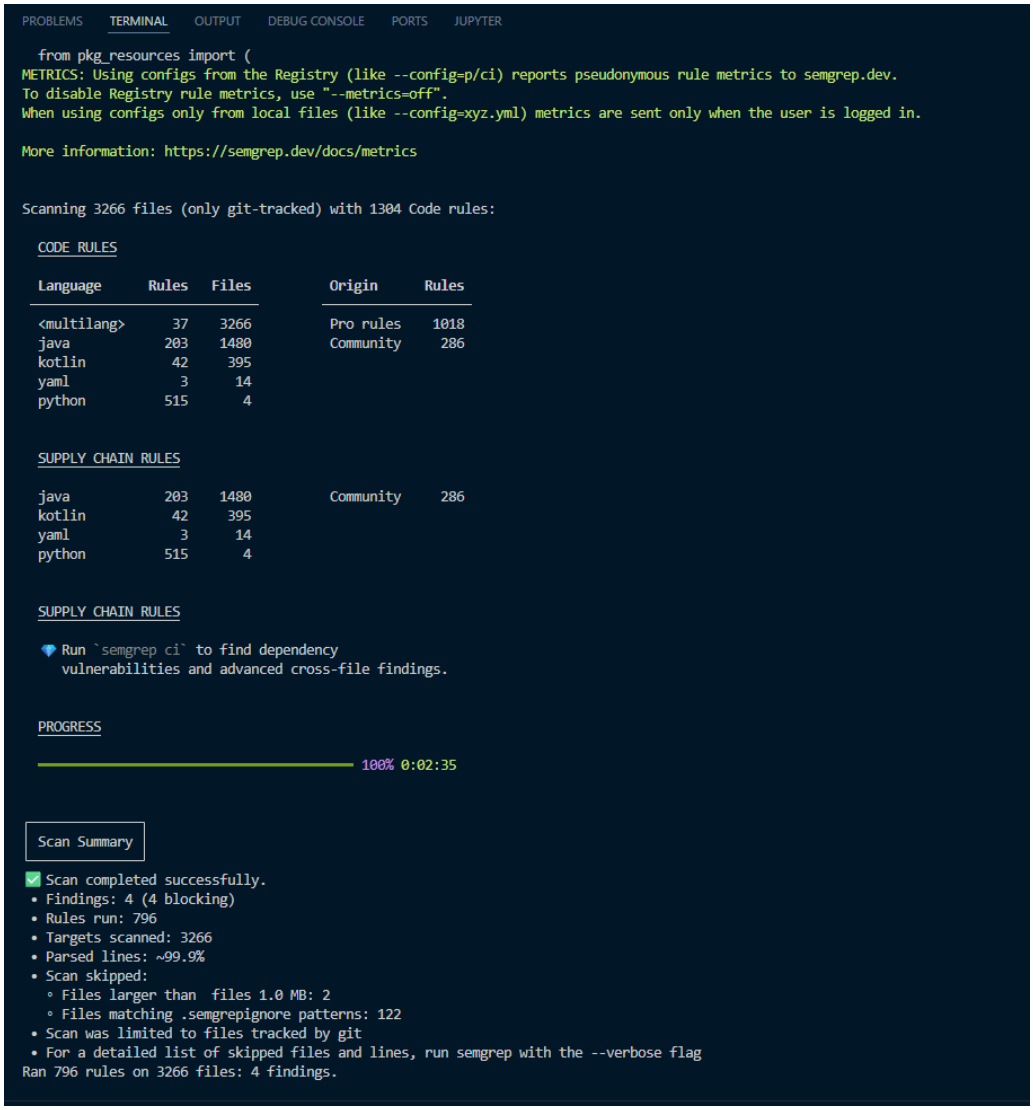

Figure 6.7: Semgrep Run for dagger

Note that CodeQL run for dagger had a very long output, so the STDOUT was stored as a file, available on the GitHub repository.

We also write a ipynb notebook to parse the SARIF files and extract the CWE IDs reported by each tool for each repository. The findings were aggregated for each project-tool combination, counting the number of occurrences for each detected CWE ID. This data was consolidated into a single CSV file with the columns: **Project_name**, **Tool_name**, **CWE_ID**, and **Num_findings**. Additionally, each CWE ID was checked against the 2024 CWE Top 25 list, and a boolean column **Is_in_top_25_CWE** was added to the csv to indicate whether the finding belongs to this critical set of vulnerabilities.

We also store a csv file to show the overall coverage of each tool across all three projects, indicating the total number of unique CWE IDs detected by each tool againt all top 25 CWEs.

Finally, we calculated the Intersection over Union (IoU) metric for each pair of tools across all three projects. The IoU was computed as the size of the intersection of the sets of CWE IDs detected by each tool divided by the size of the union of these sets. This metric tells us about the agreement and disagreement between the tools in terms of the vulnerabilities they detect. We plot these as venn diagrams for visual clarity.

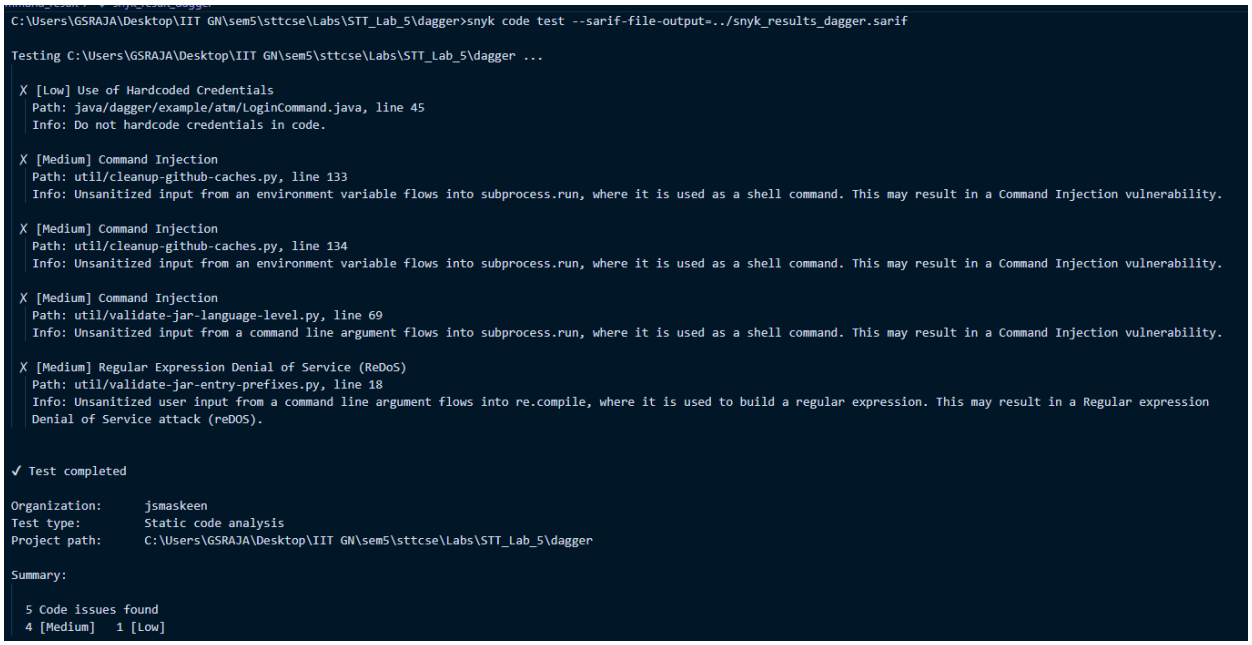

Figure 6.8: Snyk Run for dagger

6.4 Results and Analysis

We obtain the top 25 CWEs from https://cwe.mitre.org/data/csv/1430.csv.zip.

6.4.1 Pairwise findings

A sample of pairwise findings for the three tools across all three projects is shown in the table below (Table 6.1). The complete findings are available in the GitHub repository.

Project	Tool	CWE ID	Num Findings	In Top 25 CWE
dagger	CodeQL	cwe-248	2	False
dagger	CodeQL	cwe-312	14	False
dagger	Semgrep	cwe-78	3	True
dagger	Semgrep	cwe-798	1	True
dagger	Snyk	cwe-400	2	True
dagger	Snyk	cwe-78	6	True
guice	CodeQL	cwe-476	10	True
guice	CodeQL	cwe-477	4	False
guice	Semgrep	cwe-489	2	False
guice	Snyk	cwe-1004	2	False
guice	Snyk	cwe-23	2	False
sentinel	CodeQL	cwe-079	10	True
sentinel	CodeQL	cwe-117	74	False
sentinel	Semgrep	cwe-23	4	False
sentinel	Semgrep	cwe-319	2	False
sentinel	Snyk	cwe-208	2	False
sentinel	Snyk	cwe-23	14	False

Table 6.1: Sample findings (2 rows per project-tool pair).

6.4.2 Tool level CWE coverage

Table 6.2 shows the overall coverage of each tool across all three projects, indicating the total number of unique CWE IDs detected by each tool against all top 25 CWEs. The CWE IDs are truncated for readability.

Tool	CWE IDs Detected (truncated)	Top 25 CWEs Covered	Coverage (%)
CodeQL	{cwe-571, cwe-209, ...}	5	20.0
Semgrep	{cwe-79, cwe-319, ...}	3	12.0
Snyk	{cwe-79, cwe-1004, ...}	6	24.0

Table 6.2: Coverage of Top 25 CWEs by different tools (CWE list truncated for readability).

As shown in the bar chart below (Fig. 6.9), Snyk has the highest coverage of the CWE Top 25 at 24%, followed by CodeQL at 20%, and Semgrep at 12%.

To visualize the specific CWEs detected by each tool, a heatmap was generated (Fig. 6.10), showing the coverage of each tool against the CWE Top 25 list.

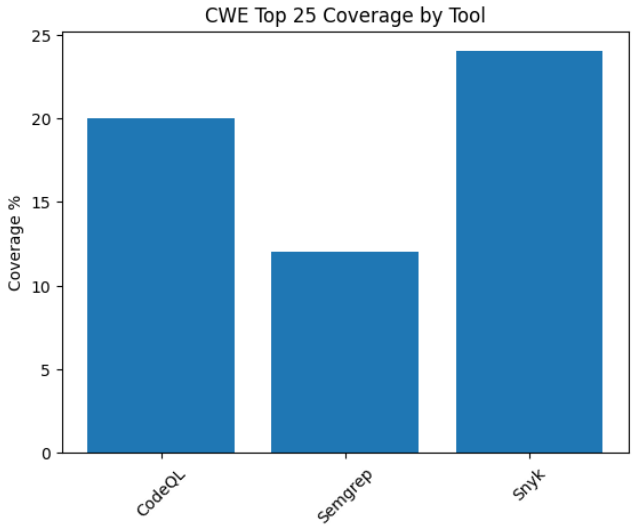

Figure 6.9: Bar chart showing coverage of Top 25 CWEs by different tools.

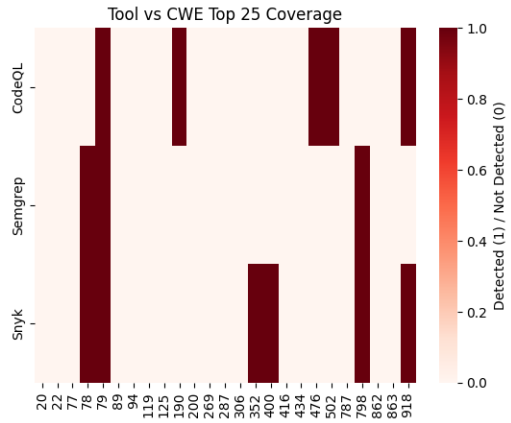

Figure 6.10: Heatmap showing coverage of each tool against the CWE Top 25 list.

6.4.3 Pairwise IoU Analysis

To understand the similarity and diversity of the tools, we computed the Intersection over Union (IoU), also known as the Jaccard Index, for each pair of tools on a per-project basis. The IoU value ranges from

0 to 1, where 1 indicates perfect agreement (the tools find the exact same set of CWEs) and 0 indicates no agreement (the tools find completely different sets of CWEs).

Dagger project

The IoU values for the dagger project are as follows:

	CodeQL	Semgrep	Snyk
CodeQL	1.000	0.000	0.000
Semgrep	0.000	1.000	0.667
Snyk	0.000	0.667	1.000

Table 6.3: IoU for dagger project.

Semgrep and Snyk show a high degree of agreement (IoU = 0.667), indicating that they detect a similar set of vulnerabilities. CodeQL, on the other hand, shows no overlap with either Semgrep or Snyk, suggesting it identifies a completely different set of weaknesses in this project. The venn diagram below visually represents this overlap (Fig. 6.11).

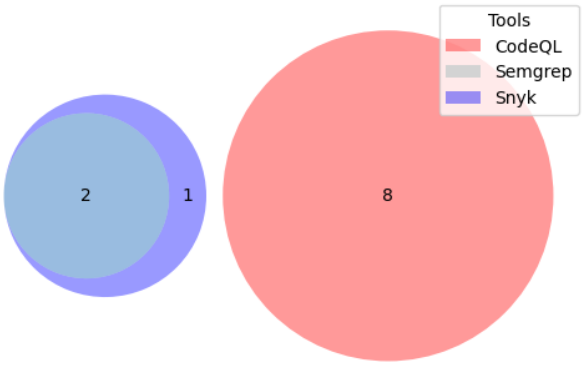

Figure 6.11: Venn diagram for dagger project.

Sentinel project

The IoU values for the sentinel project are as follows:

	CodeQL	Semgrep	Snyk
CodeQL	1.000	0.000	0.057
Semgrep	0.000	1.000	0.200
Snyk	0.057	0.200	1.000

Table 6.4: IoU for sentinel project.

In the case of the sentinel project, all three tools show very low agreement with each other, with the highest IoU being only 0.2 between Semgrep and Snyk. This indicates that the tools are largely complementary for this project, each finding a distinct set of vulnerabilities. The venn diagram below illustrates the limited overlap (Fig. 6.12).

Guice project

The IoU values for the sentinel project are as follows:

Similar to the sentinel project, the tools show low agreement for the guice project. The IoU between Semgrep and Snyk is 0.2, and there is no overlap between CodeQL and the other two tools. The venn diagram below visualizes this (Fig. 6.13).

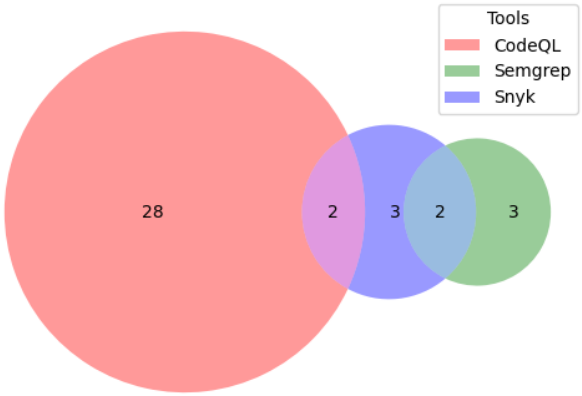

Figure 6.12: Venn diagram for sentinel project.

	CodeQL	Semgrep	Snyk
CodeQL	1.0	0.0	0.0
Semgrep	0.0	1.0	0.2
Snyk	0.0	0.2	1.0

Table 6.5: IoU for guice project.

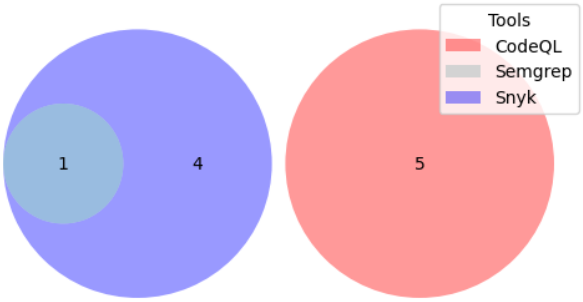

Figure 6.13: Venn diagram for guice project.

Overall IoU metrics

We consolidate the unique CWE IDs across all three projects, and the IoU values are as follows:

	CodeQL	Semgrep	Snyk
CodeQL	1.0	0.0	0.04
Semgrep	0.0	1.0	0.31
Snyk	0.04	0.31	1.0

Table 6.6: IoU over all unique CWEs detected by the three tools.

This shows that CodeQL and Semgrep or CodeQL and Snyk have very diverse detection capabilities. However, 31% of vulnerabilities detected by Snyk and Semgrep are the same. So there is partial redundancy for Snyk and Semgrep. The venn diagram below visualizes this (Fig. 6.14).

6.5 Discussion and Conclusion

This lab provided us with details about the performance of different SAST tools. Our analysis of the CWE Top 25 coverage showed that Snyk had the broadest coverage among the three tools. However, the IoU

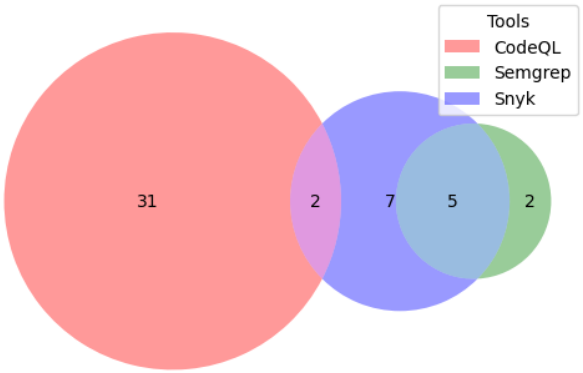

Figure 6.14: Venn diagram for overall coverage.

analysis showed that no single tool is sufficient for in depth vulnerability detection. The low IoU values across most tool pairs and projects highlight the diversity in their detection capabilities. CodeQL finds highly unique vulnerabilities, whereas there is some redundancy between Semgrep and Snyk.

To maximize CWE coverage, a combination of tools is necessary. Based on our findings, using CodeQL and Snyk (Semgrep also works but it detects lower number of vulnerabilities than Snyk) together would provide the best coverage for the analyzed projects, as they tend to find different sets of vulnerabilities. The high IoU between Semgrep and Snyk in some cases suggests that using both might lead to redundant findings.

We understood the importance of a multi-tool approach to static analysis during this lab. Each tool has its own strengths and focuses on different types of weaknesses. By combining the results from multiple tools, we can achieve a more detailed and reliable security assessment. The choice of tools should be governed by the specific programming languages, frameworks, and types of vulnerabilities that are most relevant to the project.

References

[1] MITRE / CWE, "2024 cwe top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html, accessed: 2025-09-25.

[2] W. contributors, "Jaccard index," https://en.wikipedia.org/wiki/Jaccard_index, accessed: 2025-09-25.

[3] K. Li, S. Chen, L. Fan, R. Feng, H. Liu, C. Liu, Y. Liu, and Y. Chen, "Comparison and evaluation on static application security testing (sast) tools for java," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, 2023, pp. 921–933, accessed: 2025-09-25.

[4] "Lab assignment document," Google Classroom, accessed: 2025-09-27.

Lab 7

Reaching Definitions Analyzer for C Programs

Github Repository for Lab 7

7.1 Introduction

In this lab we implement a Reaching Definitions Analyzer for C programs from scratch. We create our own custom simple parser, convert the C code into a parsed tree, then make a control flow graph from it by identifying basic blocks and their connections. Finally, we implement the reaching definitions analysis using the iterative algorithm discussed in class. Reaching Definitions Analysis is a fundamental data-flow analysis technique used in compilers to determine which definitions (assignments of values to variables) may reach a given point in the program. This is useful for optimizations such as dead code elimination and, constant propagation etc. We use an example of simple C program to illustrate the steps and output of our analysis in this document. We run the analysis on three non-trivial C programs (200-300 LOC) and store their results on the Github repository for this lab.

7.2 Setup and Tools

We have Python, and VSCode installed. `python --version` gives `Python 3.12.4` as the output. The operating system on which all commands and analysis are performed is Windows 11. We use **Graphviz** to visualize the control flow graphs, and matplotlib to plot graphs for other metrics. The Python packages used are `graphviz`, `matplotlib`, and `pandas`. These can be installed via pip. `dot -version` gives `dot - graphviz version 13.1.0 (20250701.0955)` as the output.

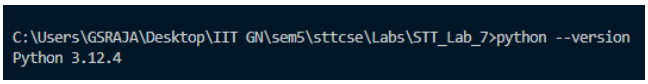

Figure 7.1: Python Version

The working directory is named `STT_Lab_7`.

7.3 Methodology and Execution

7.3.1 Program Selection

We use four C programs for our analysis. They are as follows:

1. `dft.c` (291 LOC): A program that implements the Discrete Fourier Transform. This program was chosen for its use of loops and arrays.

2. `integration.c` (250 LOC): A program that performs numerical integration along with other computations such as computing π. This program was selected for its mathematical computations, calling external procedures (considered as blackbox) and more complex control flow.

3. `labyrinth_game.c` (460 LOC): A simple game that navigates a labyrinth. This program was chosen for its interactive nature and its use of nested loops and conditionals.

4. `simple.c`: A basic program with conditional statements and variable reassignments. This program is used to provide a clear, step-by-step walkthrough of the analysis process in this document.

All programs are standalone `.c` files with a `main()` function, and the analysis is intraprocedural.

We make sure that all our programs do not have `goto` statements or `break` or `switch` statemets, as to keep the parsing and analysis simpler. Function calls to external procedures are treated as black boxes. We also ensure that the code is formatted using https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools extension in VSCode.

Throughout the parsing part, we utilise heavy regular expressions to identify various constructs in the C code. We explain the entire process of parsing and analysis using the simple.c program, which is as follows:

Initial C code:

```c
#include <stdio.h>

int random_procedure()
{
    return 23;
}

int main(void)
{
    int i = 0;
    int x = 5;
    int y = -1;

    while (i < 10)
    {
        i = i + 1;
        y = i + x;
        printf("%d", i);
        if (i > 5)
        {
            x = i;
            printf("more than 5");
        }
    }

    random_procedure();

    for (int j = 0; j < 5; j++)
    {
        int z = j + y;
        printf("%d", z);
        random_procedure();
    }
    return 0;
}
```

7.3.2 Parsing the C Code

The first step is to extract out the main function and remove blank lines and split at each newline character. This is how the program looks after this step:

```
['int i = 0;',
 'int x = 5;',
```

```
 'int y = -1;',
 'while (i < 10)',
 '{',
 'i=i+1;',
 'y = i + x;',
 'printf("%d", i);',
 'if (i > 5)',
 '{',
 'x = i;',
 'printf("more than 5");',
 '}',
 '}',
 'random_procedure();',
 'for (int j = 0; j < 5; j++)',
 '{',
 'int z = j + y;',
 'printf("%d", z);',
 'random_procedure();',
 '}',
 'return 0;']
```

Now, we convert this into a nested structure (Equivalent to a very rudimentary AST) using a recursive function. We identify blocks using curly braces, and each block is represented as a list of statements. Each statement can be a simple statement (like an assignment or a function call) or a control structure (like if, while, for) which contains its own nested list of statements. The resulting structure looks like this:

```
[{'code': 'int i = 0;', 'type': 'statement'},
 {'code': 'int x = 5;', 'type': 'statement'},
 {'code': 'int y = -1;', 'type': 'statement'},
 {'body': [{'code': 'i=i+1;', 'type': 'statement'},
           {'code': 'y = i + x;', 'type': 'statement'},
           {'code': 'printf("%d", i);', 'type': 'statement'},
           {'body': [{'code': 'x = i;', 'type': 'statement'},
                     {'code': 'printf("more than 5");', 'type': 'statement'}],
            'condition': 'i > 5',
            'type': 'if'}],
  'condition': 'i < 10',
  'loop_type': 'while',
  'type': 'loop'},
 {'code': 'random_procedure();', 'type': 'statement'},
 {'body': [{'code': 'int z = j + y;', 'type': 'statement'},
           {'code': 'printf("%d", z);', 'type': 'statement'},
           {'code': 'random_procedure();', 'type': 'statement'}],
  'condition': 'j < 5',
  'increment': 'j++',
  'initialization': 'int j = 0',
  'loop_type': 'for',
  'type': 'loop'},
 {'code': 'return 0;', 'type': 'statement'}]
```

With this, we proceed to Control Flow Graph (CFG) generation.

7.3.3 Control Flow Graph Generation

A CFG is a directed graph where each node is a basic block (a sequence of instructions with one entry and one exit) and each edge shows a possible transfer of control (normal fall-through, true/false branch, loop-back, etc.).

We define two classes, `BasicBlock` and `ControlFlowGraph`. The `BasicBlock` class represents a basic block with its statements and successors. The `ControlFlowGraph` class manages the collection of basic blocks and the start block, and a few other helper variables for Reaching Defintions Analysis.
Note: In the following procedure, we refer to the CFG class as *builder*.

Using the following way, the above rudimentary AST is converted into a CFG:

1. Entry and statements:

- The first call creates B0 (start). Every plain statement (type "statement") is appended to the current block's instructions.
- If a statement appears where a new block should start, a new BasicBlock is created and linked from the previous block.

2. If statements:
   - The builder places the `if (cond)` as an instruction in the current block.
   - Then it creates a new block for the if-body and adds an edge from the if-block to that body with label "true".
   - It also creates an after-if block (the join). The last block of the if-body is connected to the after-if block.
   - If there is an else statement, an else block is created and connected with label "false", and its last block also goes to the join. If no else is present, the builder creates a "false" edge from the if-block to the join directly.
   - Execution continues at the after-if block.
3. Loop statements:
   - For loop:
     - If an initialization exists, it is used as an instruction in the current block (Eg. `int j = 0;`)
     - A condition block is created that contains `if (condition)` and has two outgoing edges: "true" to the loop body and "false" to after-loop.
     - The loop body is a separate block; after the body, control goes to an increment block (which contains the increment code, if any), and the increment block links back to the condition block.
   - While loop:
     - The builder places the `while (cond)` as an instruction in the current block (the block acts like a condition node).
     - The "true" edge goes to the loop body; when the body finishes it links back to the condition block.
     - The "false" edge goes to the after-loop block.

The exporting of the CFG is handled separately, where each node label contains the block name plus its instructions (the code statements). The edges include optional labels like "true" / "false" (Fig. 7.2). The dot file is saved then rendered using the dot CLI (Fig. 7.3).

7.3.4 Cyclomatic Complexity Calculation

After constructing the CFG, we compute the cyclomatic complexity of the program. Cyclomatic complexity is a metric used to indicate the complexity of a program. It is the number of independent paths through a program's source code. It is calculated using the formula (for a single component):

$$E - N + 2$$

where, E is the number of edges in the graph, and N is the number of nodes in the graph. For the simple.c program, E=14, N=12, giving a cyclomatic complexity of 4.

```
digraph ControlFlowGraph {
    node [shape=box, fontname="Courier"];
    edge [fontname="Helvetica"];
    B0 [label="B0\n\lint i = 0;\lint x = 5;\lint y = -1;\l"];
    B1 [label="B1\n\lwhile (i < 10)\l"];
    B2 [label="B2\n\lrandom_procedure();\l"];
    B3 [label="B3\n\li=i+1;\ly = i + x;\lprintf(\"%d\", i);\l"];
    B4 [label="B4\n\lif (i > 5)\l"];
    B5 [label="B5\n\lx = i;\lprintf(\"more than 5\");\l"];
    B6 [label="B6\n\l\l"];
    B7 [label="B7\n\lint j = 0;\l"];
    B8 [label="B8\n\lif (j < 5)\l"];
    B9 [label="B9\n\lreturn 0;\l"];
    B10 [label="B10\n\lj++;\l"];
    B11 [label="B11\n\lint z = j + y;\lprintf(\"%d\", z);\lrandom_procedure();\l"];
    B0 -> B1;
    B1 -> B2 [label="false"];
    B1 -> B3 [label="true"];
    B2 -> B7;
    B3 -> B4;
    B4 -> B5 [label="true"];
    B4 -> B6 [label="false"];
    B5 -> B6;
    B6 -> B1;
    B7 -> B8;
    B8 -> B9 [label="false"];
    B8 -> B11 [label="true"];
    B10 -> B8;
    B11 -> B10;
}
```

Figure 7.2: Dot File for the CFG of simple.c

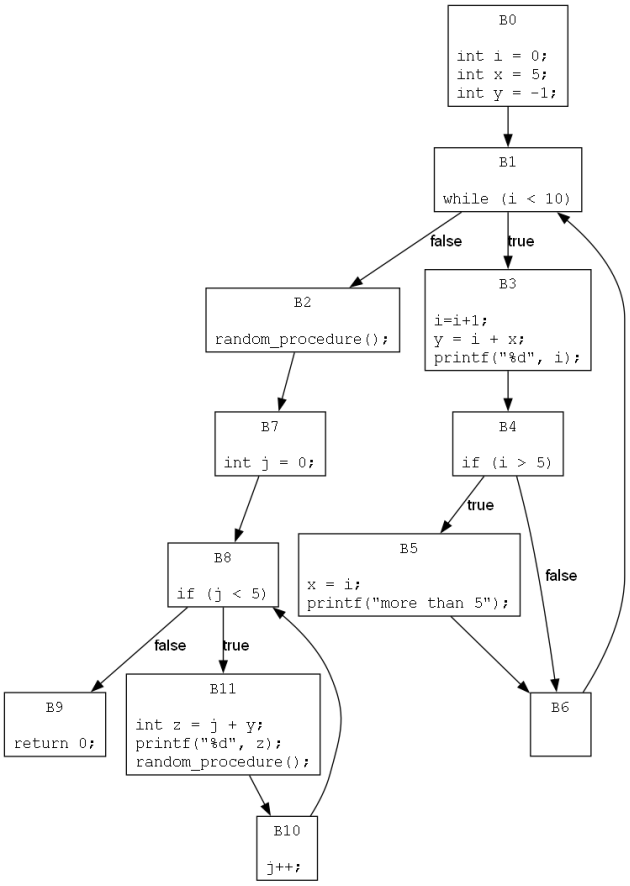

Figure 7.3: Rendered dot File for the CFG of simple.c

7.3.5 Reaching Definitions Analysis

Reaching Definitions Analysis determins, for each point in the program, which definitions (assignments of values to variables) can reach that point. The program points are the entry or exit of each basic block in the CFG.

A definition is any assignment statement, and we use the following regex to find all such assignment statemets. Fig 7.4 shows which sort of assignment statements can be matched by our regular expression.

```
1    re.match(r"\s*(?:[\w\s]+\s+)?\*?(\w+)\s*\[?.*?]?\s*=\s*.+;|(\w+)\+\+|(\w+)
     --|\+\+(\w+)|--(\w+)")
```

The dataflow equations for Reaching Definitions Analysis are as follows:

$$in[B] =_{P \in pred(B)} out[P]$$

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

Where:

- $in[B]$ is the set of definitions reaching the entry of block B.
- $out[B]$ is the set of definitions reaching the exit of block B.
- $gen[B]$ is the set of definitions generated within block B.
- $kill[B]$ is the set of definitions that are killed (overwritten) by block B.
- $pred(B)$ is the set of predecessor blocks of B in the CFG.

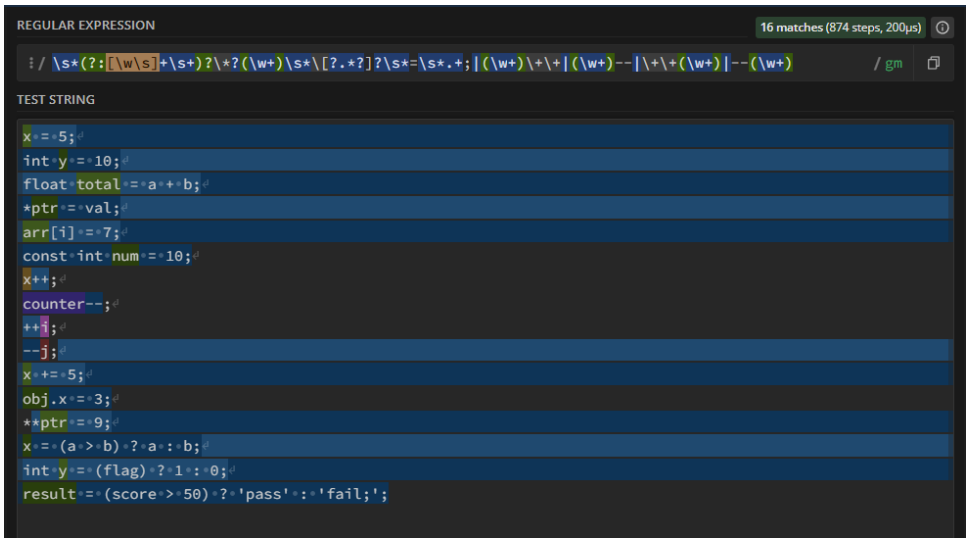

Figure 7.4: Example of the assignment statements our regex can match.

These equations are applied iteratively until the in and out sets converge (i.e., they no longer change). Initially, all in[B] sets and out[B] sets are initialized to ϕ.

The result for the interative analysis per iteration is provided as a CSV file on the Github repository for this lab. The final in and out sets for simple.c are as follows:

Basic-Block	gen[B]	kill[B]	in[B]	out[B]
B0	['D1', 'D2', 'D3']	['D4', 'D5', 'D6']	[]	[D1, D2, D3]
B1	[]	[]	[D1, D2, D3, D4, D5, D6]	[D1, D2, D3, D4, D5, D6]
B2	[]	[]	[D1, D2, D3, D4, D5, D6]	[D1, D2, D3, D4, D5, D6]
B3	['D4', 'D5']	['D1', 'D3']	[D1, D2, D3, D4, D5, D6]	[D2, D4, D5, D6]
B4	[]	[]	[D2, D4, D5, D6]	[D2, D4, D5, D6]
B5	['D6']	['D2']	[D2, D4, D5, D6]	[D4, D5, D6]
B6	[]	[]	[D2, D4, D5, D6]	[D2, D4, D5, D6]
B7	['D7']	['D8']	[D1, D2, D3, D4, D5, D6]	[D1, D2, D3, D4, D5, D6, D7]
B8	[]	[]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]
B9	[]	[]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]
B10	['D8']	['D7']	[D1, D2, D3, D4, D5, D6, D7, D8, D9]	[D1, D2, D3, D4, D5, D6, D8, D9]
B11	['D9']	[]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]	[D1, D2, D3, D4, D5, D6, D7, D8, D9]

Figure 7.5: Final DFA iteration for simple.c

where the corresponding definitions are:

```
{
  "D1": ["i", "int i = 0;"],
  "D2": ["x", "int x = 5;"],
  "D3": ["y", "int y = -1;"],
  "D4": ["i", "i = i + 1;"],
  "D5": ["y", "y = i + x;"],
  "D6": ["x", "x = i;"],
  "D7": ["j", "int j = 0;"],
  "D8": ["j", "j++;"],
  "D9": ["z", "int z = j + y;"]
}
```

7.3.6 Multiple Reaching Definition Identification based on in[B] and out[B] sets

For each block, using in[B] and out[B] we can tell if some variable has multiple reaching definitions at a point, by checking if there are multiple defintions reaching that block for some of its variables. For

example, in block B7, we see that the in[B7] set contains D1 and D4 for variable i, indicating that at the start of block B7, variable i can have two possible definitions reaching it: the initial definition (D1) and the updated definition from within the loop (D4). This indicates that there are multiple paths in the program that can lead to block B7, each with a different value for i.

For `simple.c`, the blocks with multiple reaching definitions are:


```
{"B1":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"]},
 "B2":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"]},
 "B3":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"]},
 "B4":{"x":["D2","D6"]},
 "B5":{"x":["D2","D6"]},
 "B6":{"x":["D2","D6"]},
 "B7":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"]},
 "B8":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"],"j":["D7","D8"]},
 "B9":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"],"j":["D7","D8"]},
 "B10":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"],"j":["D7","D8"]},
 "B11":{"i":["D1","D4"],"x":["D2","D6"],"y":["D3","D5"],"j":["D7","D8"]}
}
```


7.4 Results and Analysis

We run the analysis on the three non-trivial C programs mentioned earlier. The results are stored on the Github repository for this lab.

The cyclomatic complexities for the three programs are shown in Table 7.1.

Program	No. Of Edges (**E**)	No. Of Nodes (**N**)	Cyclomatic Complexity (**CC**)
`dft.c`	92	73	21
`integration.c`	179	147	34
`labyrinth_game.c`	271	201	72

Table 7.1: CC metrics for the chosen C program.

The iterations for the Reaching Definitions Analysis for these programs are provided in the Github repository. Some interesting observations are as follows:

Fig 7.6 shows the cyclomatic complexity vs (program, LOC) for the three programs. We observe that as the lines of code increase, the cyclomatic complexity also tends to increase, indicating more complex control flow in larger programs.

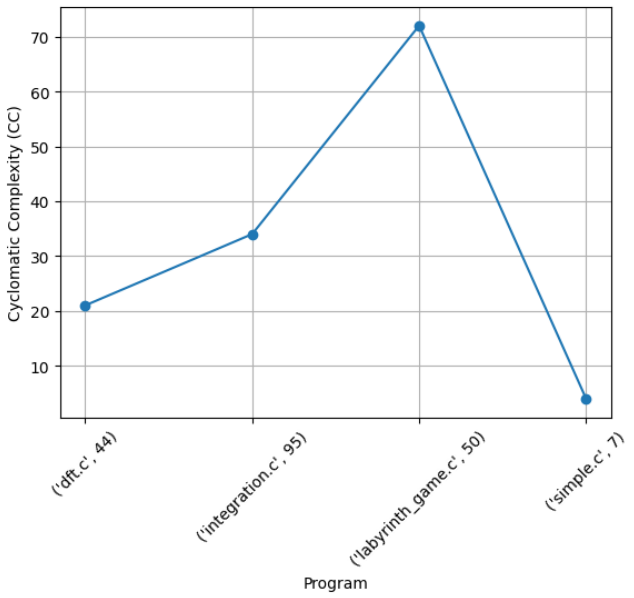

Figure 7.6: CC vs program

We then plot in the following graphs, the number of changes in In[B] and Out[B] sets per iteration for the four programs (Fig 7.7, Fig 7.8, Fig 7.9 and, Fig 7.10).

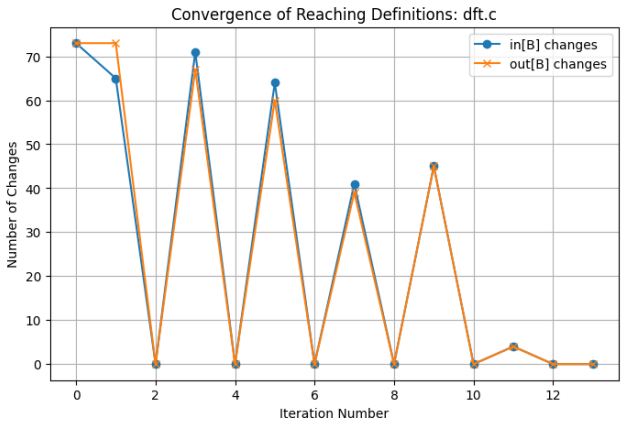

Figure 7.7: `dft.c`: Changes in in[B] and out[B] per iteration

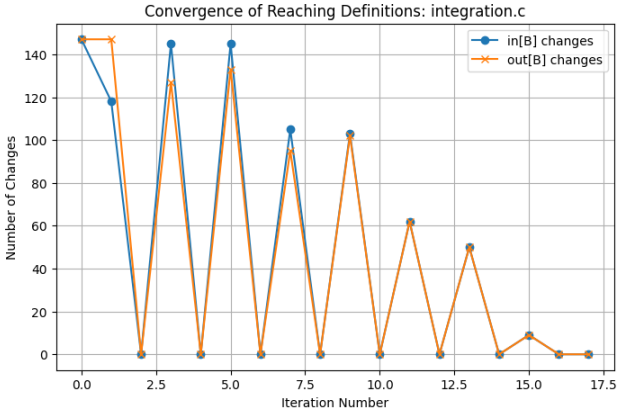

Figure 7.8: `integration.c`: Changes in in[B] and out[B] per iteration

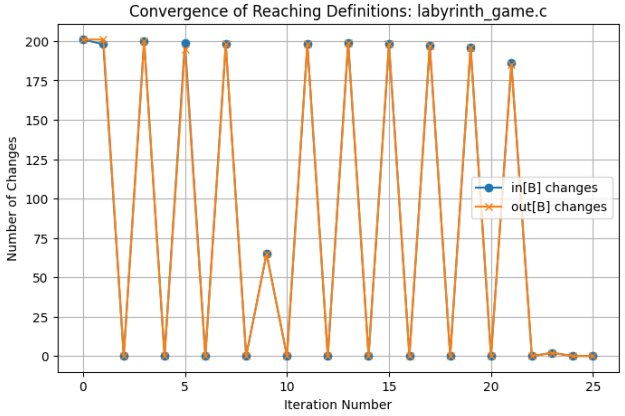

Figure 7.9: `labyrinth_game.c`: Changes in in[B] and out[B] per iteration

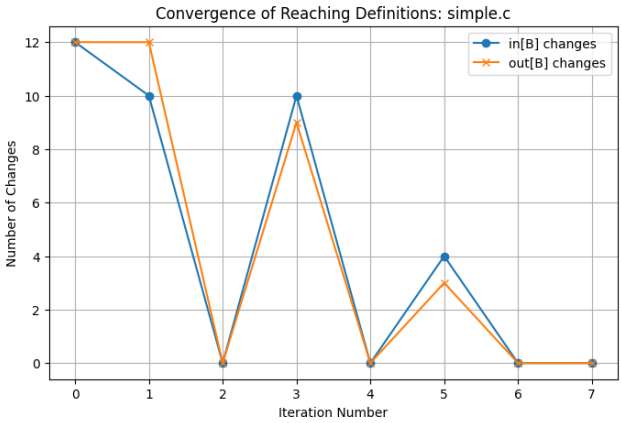

Figure 7.10: `simple.c`: Changes in in[B] and out[B] per iteration

We observe that in all four programs, overall the number of changes decreases with each iteration, indicating convergence of the analysis. However, the number of changes are alternating between some decreasing value and zero. This alternating pattern of changes in in[B] and out[B] occurs due to the forward propagation of reaching definitions. In each iteration, a block's in[B] is updated from the out[B] sets of its predecessors, and then its out[B] is updated based on its in[B] and its gen/kill sets. Therefore, a change in out[B] in one iteration triggers a change in successor in[B] in the next iteration, and so on, producing a wave-like alternation until the analysis converges.

7.5 Discussion and Conclusion

In this lab, we implemented and understand the principles of Reaching Definitions Analysis, by constructing CFGs and applying data flow equations to chosen C programs in increasing complexity. The analysis of the four programs, from the simple simple.c to the more complex labyrinth_game.c, showed the correctness and scalability of our custom `CFGMaker1000`.

Through constructing the control flow graph and performing reaching definitions analysis, we were able to identify, for each basic block, the definitions of variables that may reach it. The results clearly show that multiple definitions can reach the same block for a given variable, depending on the path taken through the program. This analysis reveals all possible definitions of variables that can be in effect at a

program point, showing the effect of control flow and multiple execution paths. This information is useful for compiler level optimzations such as constant propagation, and dead code elimination, since we can determine at compile time if some variable has a single definition or multiple definitions at some point.

References

[1] T. G. Authors. (2024) Dot language | graphviz. Accessed: 2024-09-30. [Online]. Available: https://graphviz.org/doc/info/lang.html

[2] ——. (2021) Graphviz. Accessed: 2024-09-30. [Online]. Available: https://graphviz.org/

[3] U. Khedker, "Cs618: Program analysis (2020-2021)," Course homepage, Dept. of Computer Science, IIT Bombay, 2020, accessed: 2024-09-30. [Online]. Available: https://www.cse.iitb.ac.in/~uday/courses/cs618-20/

[4] "Lab assignment document," Google Classroom, accessed: 2025-09-30.

[5] D. R. C. S. of Computer Science. (2025) Control flow graphs. Accessed: 2025-09-30. [Online]. Available: https://www.cs.toronto.edu/~david/course-notes/csc110-111/17-graphs/08-control-flow-graphs.html

[6] GeeksforGeeks. (2025) Control flow graph (cfg) - software engineering. Accessed: 2025-09-30. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/software-engineering-control-flow-graph-cfg/

[7] "regex101: build, test, and debug regular expressions," https://regex101.com/, accessed: 2025-09-30.